



FSSM

Filière : IAP

<title>

RAPPORT D'ANALYSE ET DE BONNES PRATIQUES

</title>

Projet Sitio Agency — Site Web Statique

HTML · CSS · JavaScript | Mai 2026

Réalisé par les étudiantes :

Siham Hadry (G145013453) & Sara Farrah (G139121450)

Encadré par la professeur : Soukaina MJAHER

1. Introduction

Ce rapport analyse le code source du projet Sitio Agency, une agence digitale fictive présentée sous la forme d'un site web statique à page unique (SPA simulée). L'objectif est d'évaluer la conformité des fichiers `index.html`, `style.css` et `script.js` aux bonnes pratiques du développement web moderne, et de proposer des axes d'amélioration concrets pour garantir la maintenabilité, l'accessibilité, la performance et la sécurité du projet.

2. Description du Projet

2.1 Architecture générale

Le projet adopte une architecture SPA (Single Page Application) simulée en HTML/CSS/JS pur, sans framework back-end ni bundler. Cinq pages virtuelles sont gérées par un routeur JavaScript minimal (`showPage`) qui affiche/masque des blocs `<div>` via des classes CSS. Les fichiers sont structurés comme suit :

- `index.html` — balisage sémantique et contenu des 5 sections (Accueil, Services, Portfolio, À propos, Contact).
- `style.css` — feuille de styles unique (~700 lignes) avec variables CSS, responsive design et animations.
- `script.js` — logique d'interaction : navigation SPA, filtres portfolio, FAQ accordéon, compteurs animés, formulaire, curseur personnalisé.

2.2 Fonctionnalités clés

- Navigation SPA sans rechargement de page.
- Filtrage du portfolio par catégorie (Sites Web, E-commerce, Applications, Design).
- Accordéon FAQ avec animation CSS `max-height`.
- Formulaire de contact avec validation côté client et retour visuel de succès.
- Scroll reveal via `IntersectionObserver`.
- Compteurs animés déclenchés à l'entrée de la page « À propos ».
- Curseur personnalisé (`cursor-dot`).
- Navbar avec effet de fond au défilement (classe `.scrolled`).

3. Analyse des Bonnes Pratiques

3.1 Structure HTML

✓ POINTS POSITIFS

- Utilisation cohérente de balises sémantiques : `<nav>`, `<section>`, `<footer>`.
- Déclaration correcte du DOCTYPE et de l'encodage UTF-8.
- Balise `<meta name="viewport">` présente pour le responsive.
- Chargement des polices Google Fonts et Bootstrap via CDN avec lien canonique.

POINTS À AMÉLIORER

- Absence de balises `<main>` : le contenu principal devrait être enveloppé dans `<main>` pour améliorer l'accessibilité et le référencement.
- Attributs alt manquants : bien qu'aucune vraie image ne soit utilisée, les emojis décoratifs (, ...) ne portent pas d'attribut `aria-hidden="true"`, ce qui peut perturber les lecteurs d'écran.
- Handlers inline (`onclick="..."`) dans le HTML : cette pratique couple le balisage et la logique JavaScript. Il est préférable d'utiliser `addEventListener` dans `script.js` pour une meilleure séparation des responsabilités.
- Pas de balise `<title>` descriptive par page : une seule valeur statique est définie pour l'onglet du navigateur, indépendamment de la section visible.
- Absence de balises `<meta description>` et Open Graph, utiles pour le SEO et le partage sur les réseaux sociaux.
- L'attribut `lang="fr"` est correct mais devrait être complété par `hreflang` si une version multilingue est envisagée.

3.2 Feuille de Styles (CSS)

✓ POINTS POSITIFS

- Utilisation de variables CSS (`:root`) pour les couleurs et dimensions clés (`--blue`, `--black`, `--nav-h...`) : facilite la maintenance et la cohérence visuelle.
- Approche Mobile-First avec des breakpoints `@media` bien définis (1024 px, 768 px).
- Emploi de `clamp()` pour la typographie fluide (`font-size: clamp(2rem, 4vw, 3rem)`), ce qui garantit une lisibilité optimale sur toutes les tailles d'écran.
- Transitions et animations légères (`opacity`, `transform`, `max-height`) plutôt que des propriétés coûteuses comme `width` ou `height`.
- Classe `.reveal` avec `IntersectionObserver` : bonne pratique pour les animations au défilement, plus performante que les listeners scroll.
- Reset minimal (*) et `box-sizing: border-box` appliqués globalement.

POINTS À AMÉLIORER

- Fichier CSS unique de ~700 lignes : pour un projet de cette envergure, une organisation par composants (BEM, SMACSS) ou l'utilisation d'un pré-processeur (SASS/SCSS) améliorerait la lisibilité et la maintenabilité.
- Valeurs codées en dur pour les couleurs de catégories de projet (background: linear-gradient(135deg,#0a1628,#0d2040)) au lieu d'utiliser les variables CSS existantes.
- La propriété color: var(--gray-700, #2a3548) utilise un fallback inline pour une variable non déclarée dans :root. Il convient d'ajouter --gray-700 à la liste des variables officielles.
- Absence de prefers-reduced-motion : les utilisateurs sensibles aux animations devraient pouvoir désactiver les transitions (@media (prefers-reduced-motion: reduce)).
- Le curseur personnalisé (#cursor-dot) n'a pas de comportement de repli si JavaScript est désactivé.

3.3 JavaScript

✓ POINTS POSITIFS

- Code fonctionnel et lisible, organisé en fonctions nommées, chacune ayant un rôle clair (showPage, filterProjects, toggleFaq...).
- Utilisation de IntersectionObserver pour le scroll reveal et unobserve() après la première détection : évite les fuites mémoire.
- La validation du formulaire vérifie les champs requis avant envoi et fournit un retour visuel (bordure rouge).
- Les compteurs animés sont déclenchés uniquement à l'entrée de la page « À propos », évitant une exécution inutile.

POINTS À AMÉLIORER

- Absence de 'use strict' : l'activation du mode strict détecte les erreurs silencieuses et améliore la robustesse du code.
- Fonctions globales exposées au window (showPage, filterProjects...) : en production, encapsuler dans un module IIFE ou utiliser des modules ES6 (import/export) évitera les collisions de noms.
- Validation email superficielle : le code vérifie uniquement qu'un champ email n'est pas vide, sans tester le format avec une expression régulière.
- Usage de alert() pour signaler les erreurs de formulaire : cette méthode native est intrusive. Un message d'erreur inline serait plus accessible.
- innerHTML utilisé dans submitForm() avec du HTML statique : bien que sans données utilisateur ici, cette habitude est risquée. Préférer createElement ou textContent quand c'est possible.
- Pas de gestion d'erreur sur l'appel simulé de formulaire : en préparation d'une connexion backend réelle, un bloc try/catch s'impose.

4. Accessibilité (A11Y)

L'accessibilité est un aspect fondamental souvent négligé sur les sites vitrines. Voici les constats pour le projet Sitio :

POINTS À AMÉLIORER

- Pas de gestion du focus clavier : la navigation SPA via `showPage()` ne déplace pas le focus vers le haut de la nouvelle « page », ce qui désoriente les utilisateurs au clavier ou aux technologies d'assistance.
- Éléments interactifs sans rôle ARIA : les boutons de filtre (`.filter-btn`) et les questions FAQ (`.faq-q`) devraient porter `aria-expanded` et `aria-controls`.
- Contraste des couleurs : la combinaison `--gray-400` (`#7A8EAD`) sur fond `--gray-900` (`#0F1624`) peut ne pas satisfaire le ratio WCAG AA de 4.5:1 pour le texte courant.
- Menu mobile (`.mobile-menu`) non annoncé aux lecteurs d'écran lors de son ouverture : ajouter `aria-expanded` sur le bouton burger et `aria-hidden` sur le menu.

5. Performance

✓ POINTS POSITIFS

- Pas d'images réelles : l'utilisation d'emojis et de blocs CSS pour les visuels élimine le poids des fichiers images.
- Animations CSS utilisant `transform` et `opacity`, qui sont composées sur GPU sans déclencher de reflow.
- Bootstrap chargé depuis un CDN, ce qui peut profiter du cache partagé entre sites.

POINTS À AMÉLIORER

- Bootstrap complet importé alors que seules quelques classes utilitaires (`container`, `btn`) sont utilisées. Un extrait minimaliste ou l'abandon total de Bootstrap réduirait le poids réseau de ~30 Ko.
- Google Fonts chargées sans `display=swap` : ajouter `&display=swap` évite le flash de texte invisible (FOIT) lors du chargement.
- Absence de mise en cache (headers `Cache-Control`) et de compression (`gzip/brotli`) : à configurer côté hébergeur pour accélérer les chargements suivants.
- Le CSS et le JS pourraient être minifiés pour la mise en production (outils : Terser pour JS, cssnano pour CSS).

6. Référencement (SEO)

✓ POINTS POSITIFS

- Hiérarchie des titres respectée (H1 unique par page, H2 pour les sections, H3 absent).
- Contenu textuel riche et structuré, favorable à l'indexation.
- Utilisation de balises sémantiques (`<nav>`, `<section>`, `<footer>`).

POINTS À AMÉLIORER

- Pas de balise `<meta name="description">` : les moteurs de recherche génèrent alors automatiquement un extrait, souvent moins pertinent.

- Pas de données structurées (JSON-LD Schema.org) pour l'organisation, les services ou les avis clients : manque à gagner en résultats enrichis Google.
- Le routage SPA côté client sans rendu serveur (SSR) rend les URLs non indexables par les robots. Pour un site statique réel, envisager une génération statique (Astro, Eleventy) ou un sitemap.xml.
- Absence de fichier robots.txt et sitemap.xml.

7. Sécurité

En tant que site statique sans backend, la surface d'attaque est réduite. Néanmoins, quelques points méritent attention :

- Formulaire sans protection CSRF ni honeypot : si le formulaire est connecté à un service tiers (Formspree, Netlify Forms...), s'assurer que le fournisseur gère la protection contre le spam.
- innerHTML dans submitForm() : même avec du contenu statique, documenter clairement que les données utilisateur ne doivent jamais être insérées via innerHTML sans assainissement préalable.
- Absence d'en-têtes de sécurité HTTP (Content-Security-Policy, X-Frame-Options) : à configurer au niveau du serveur web ou de l'hébergeur (Netlify, Vercel...).
- Les CDN tiers (Bootstrap, Google Fonts) devraient utiliser l'attribut integrity (Subresource Integrity) pour garantir l'intégrité des fichiers.

8. Tableau Récapitulatif

Domaine	Niveau actuel	Priorité d'amélioration
Structure HTML	Bon	Moyenne
CSS & Variables	Très bon	Faible
JavaScript	Bon	Moyenne
Accessibilité	À améliorer	Haute
Performance	Bon	Moyenne
SEO	Moyen	Haute
Sécurité	Acceptable	Moyenne

9. Recommandations Prioritaires

COURT TERME (1-2 JOURS)

- Ajouter <main> autour du contenu principal et aria-hidden="true" sur les emojis décoratifs.
- Remplacer les handlers onclick inline par des addEventListener dans script.js.
- Ajouter &display=swap au lien Google Fonts.

- Déclarer `--gray-700` dans `:root` et remplacer les couleurs codées en dur des projets portfolio.
- Ajouter `@media (prefers-reduced-motion: reduce)` pour désactiver les animations.

MOYEN TERME (1-2 SEMAINES)

- Ajouter `aria-expanded` et `aria-controls` sur le menu burger et les éléments FAQ.
- Implémenter une validation email avec RegEx dans `submitForm()`.
- Ajouter les balises `<meta description>` et Open Graph.
- Vérifier les ratios de contraste avec un outil comme WebAIM Contrast Checker.
- Minifier CSS et JS pour la mise en production.

LONG TERME (SELON ÉVOLUTION DU PROJET)

- Migrer vers un générateur de site statique (Astro, Eleventy) pour le SEO et la maintenabilité.
- Remplacer Bootstrap complet par un extrait minimal ou un système utilitaire léger (Open Props).
- Ajouter des données structurées JSON-LD (LocalBusiness, Service, Review).
- Configurer les en-têtes HTTP de sécurité via l'hébergeur.

10. Conclusion

Le projet Sitio Agency constitue une base de qualité pour un site web statique d'agence digitale. Le code est lisible, bien organisé et adopte des pratiques modernes telles que les variables CSS, les animations GPU-friendly et l'IntersectionObserver. Les axes d'amélioration identifiés — accessibilité, SEO et séparation des responsabilités HTML/JS — ne remettent pas en cause la solidité globale du projet, mais sont essentiels pour répondre aux standards professionnels et aux attentes des moteurs de recherche et des utilisateurs en situation de handicap. En appliquant les recommandations prioritaires listées dans ce rapport, le projet atteindra un niveau de qualité production élevé, maintenable et évolutif.